

HarborLens 6:00 A.M. Cutover

Table of Contents

1. [Learning Objectives](#)
2. [Case Pipeline](#)
3. [The run looked green. Maya stopped the cutover anyway.](#)
4. [What the team had already tried](#)
5. [The exhibits that changed the meeting](#)
6. [Questions before you read the verdict](#)
7. [Model answers Maya would accept](#)
8. [The architecture Maya pushed through](#)
9. [Take the lesson somewhere else](#)
10. [Decision Points](#)
11. [Analysis Walkthrough](#)

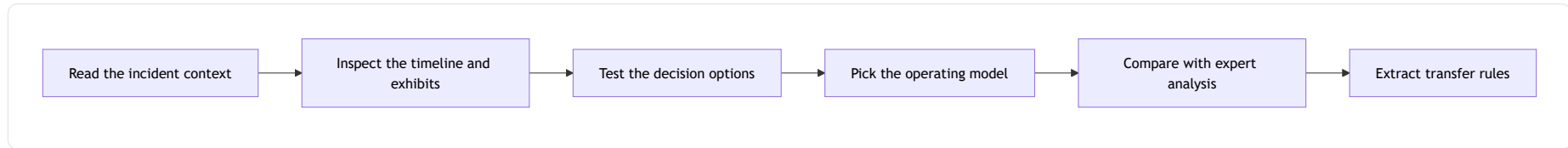
 Duration: 30 minutes

Learning Objectives

- Diagnose why an AI-native ingestion pipeline can look successful while producing untrustworthy downstream data.
- Choose a storage and transformation pattern that preserves source evidence while still serving AI-ready curated outputs.
- Defend an orchestration and rerun design using concrete recovery, lineage, and governance requirements.
- Compare ingestion methods and freshness targets against actual source behavior, consumer needs, and operating

constraints.

Case Pipeline



The run looked green. Maya stopped the cutover anyway.

Sep 20, 2024, 5:47 a.m. Maya Chen, staff data engineer at HarborLens, was staring at two screens that disagreed. Airflow showed the new shipment intelligence pipeline as successful. The launch Slack channel did not feel successful. Exception managers were reporting that the retrieval assistant could not explain delays for three West Coast lanes, and the morning dashboard was undercounting scanned handoffs by 15%. At 6:00 a.m., HarborLens was supposed to switch 220 dispatch users from a brittle legacy feed to this AI-native platform.

The pipeline's mission sounded straightforward: ingest overnight shipment events from a `Carrier API`, ingest partner-delivered `CSV files` with depot scans, preserve the original extracts, transform them into a trusted daily operations product, and publish a cleaned corpus for retrieval over delivery notes and PDFs. The stakes were not abstract. If Maya cut over on bad data, operations managers would escalate drivers unnecessarily, customer support would quote wrong statuses, and the first monthly training snapshot for ETA prediction would be poisoned before the model team even started feature work.

The precise decision on the table: ship the simplified single-path pipeline today, or delay cutover and move to a source-split design with restricted raw storage, hybrid transformations, documented curated products, and explicit rerun controls.

What the team had already tried

- **2024-07-29** — HarborLens defined the use case: daily operations dashboard by 6:00 a.m., retrieval assistant for exception

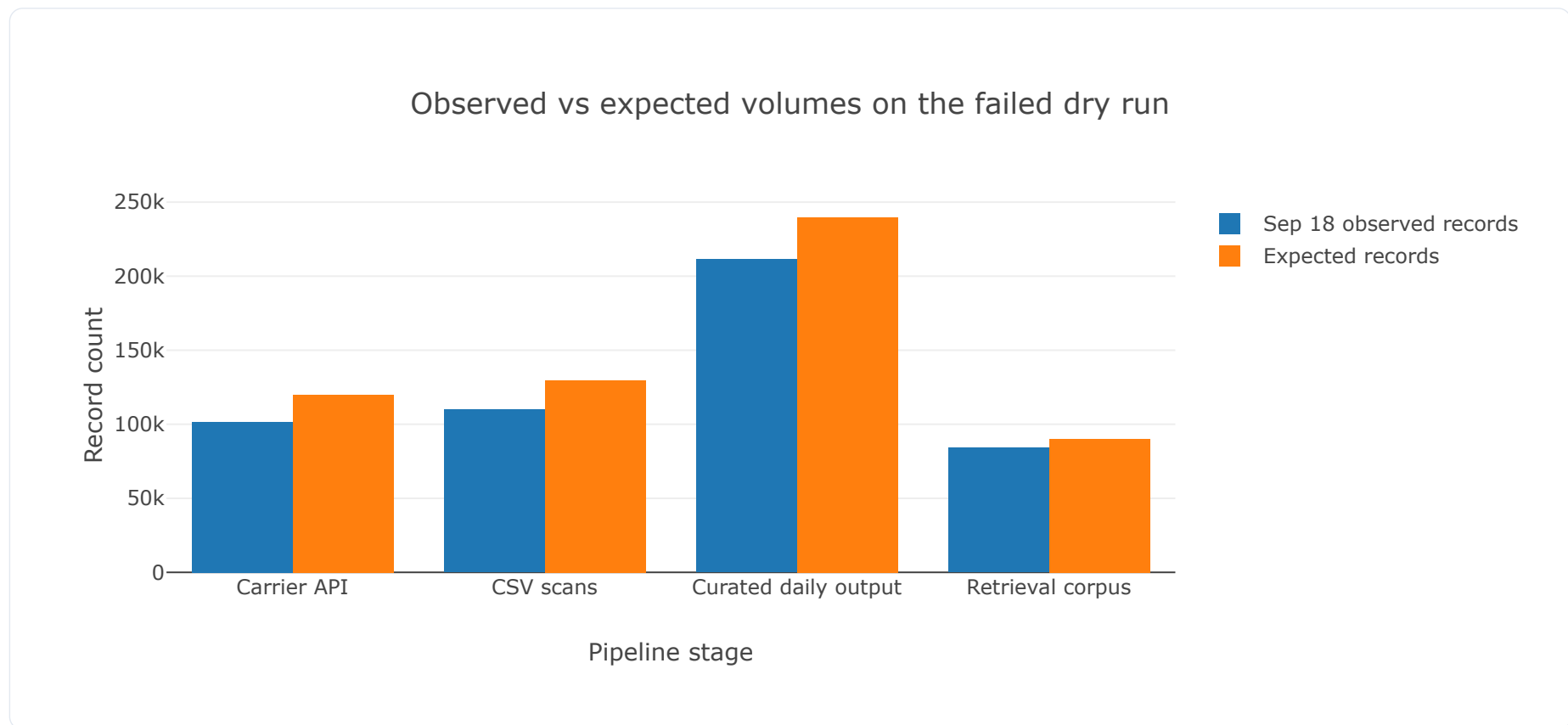
managers by 6:15 a.m., and a static monthly training snapshot for ETA modeling.

- **2024-08-05** — A prototype `Python` notebook pulled both the `Carrier API` and partner `CSV files` into one warehouse table; it was fast enough for demos but had no run manifest or raw retention.
- **2024-08-19** — Source assessment found partner CSV deliveries usually arrive between **5:20-5:35 a.m.**, while the Carrier API paginates 2,000 rows per page and occasionally returns HTTP 200 with incomplete page sets during peak traffic.
- **2024-08-28** — The analytics engineer added `SQL` transforms and a few checks for null status codes, but reruns still appended duplicate rows because the load lacked a run boundary.
- **2024-09-09** — The team introduced `dbt` for curated models and basic tests, yet continued reading directly from the merged landing table because raw source-aligned storage had not been created.
- **2024-09-18** — A dry run passed all orchestration tasks in **24 minutes**, but downstream totals were low by **18,406 rows** after a partial API result set was treated as complete.
- **2024-09-20** — Cutover review opened with one question from Maya: do we trust this pipeline enough to let an assistant answer from it?

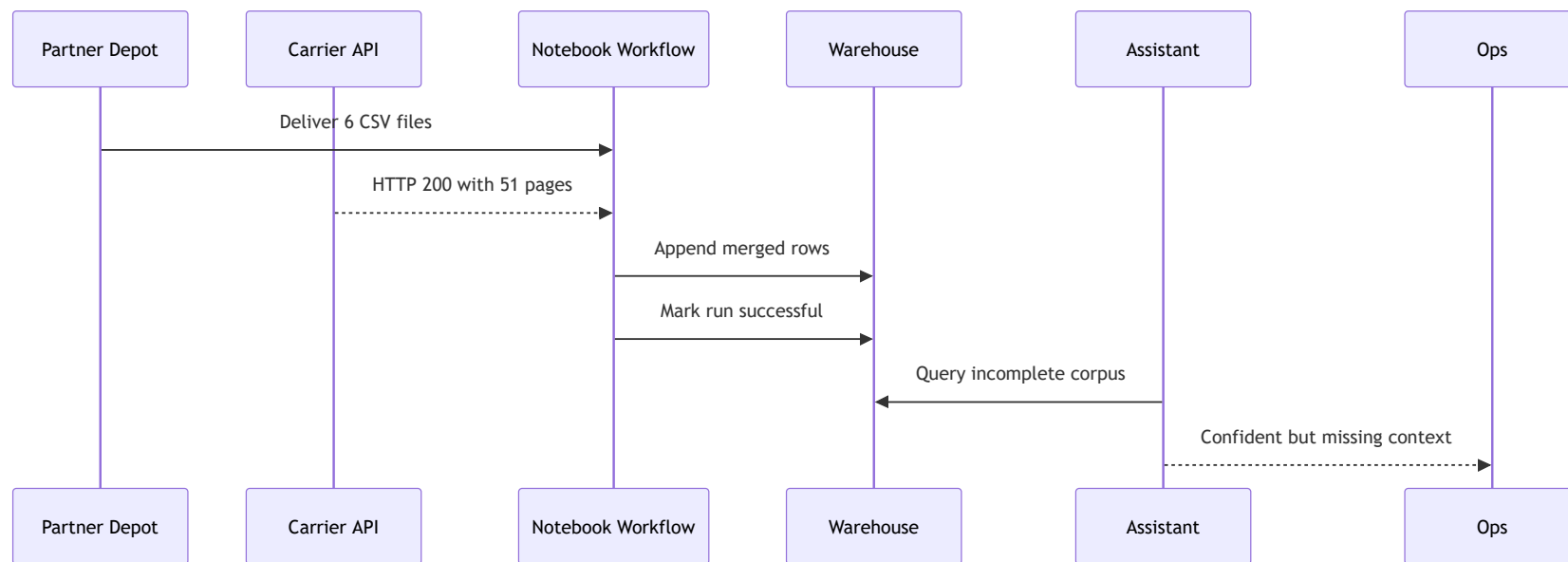
The exhibits that changed the meeting

Exhibit	Source	Expected behavior	Actual Sep 18 behavior	Why it mattered
A	Carrier shipment events API	~120,000 rows by 5:40 a.m.; 60 pages of 2,000 rows	HTTP 200 after 51 pages; 101,571 rows landed	The load looked successful without proving completeness
B	Partner depot scans CSV	6 files, 18k-22k rows each, filename per depot/date	One file was 0 bytes but still copied to landing	Empty file was not treated as a failed business condition
C	Curated	One row per shipment-	Rerun appended 7,942	No parameterized rerun

	shipment_events_daily	stop-event, deduped by source keys	duplicates on Sep 17 retry	boundary
D	Retrieval corpus	Sanitized delivery notes and approved PDF snippets only	Raw notes with phone numbers flowed through	Governance labeling happened too late
E	Consumer freshness contract	Dashboard daily; assistant daily by 6:15; training snapshot monthly static	One pipeline served all three with one freshness assumption	Consumer needs were collapsed into a single path



Maya also sketched the current control failure as a sequence problem, not just a data problem.



Questions before you read the verdict

- **[Analyze]** Which exhibit most strongly rules out shipping the single merged landing-table design, and what exact signal makes it disqualifying rather than merely imperfect?
- **[Evaluate]** Given the three consumers, where should HarborLens use daily refresh, where should it require strict completeness, and where should it publish a static versioned snapshot?
- **[Apply]** What minimum run evidence would you require per source so a 6:10 a.m. on-call engineer can prove whether the run is safe to rerun?
- **[Evaluate]** Should the platform fail fast on the empty CSV and partial API response, or quarantine-and-continue for any part of the pipeline? Be specific about which products can tolerate that choice.
- **[Apply]** If you were rewriting the implementation this week, which parts belong in `Python`, which in `SQL / dbt`, and what orchestration layer replaces the notebook?

Model answers Maya would accept

The strongest disqualifier is Exhibit A paired with Exhibit C. The partial `Carrier API` result returned **101,571** rows instead of roughly **120,000**, yet the run still marked success, and the Sep 17 retry appended **7,942 duplicates**. Together they show the pipeline cannot prove completeness or rerun safely. That is not a tuning issue; it is a trust boundary failure, which points away from Decision Option 1 and toward Option 2.

Freshness should be split by product contract. The dashboard can remain **daily** as long as incomplete lanes are flagged. The retrieval assistant also refreshes daily, but it needs stricter completeness checks because incomplete context degrades answer safety. The training product should be a **static monthly snapshot** with versioned inputs and documented exclusions. A single freshness rule for all three, as in Exhibit E, is the design mistake.

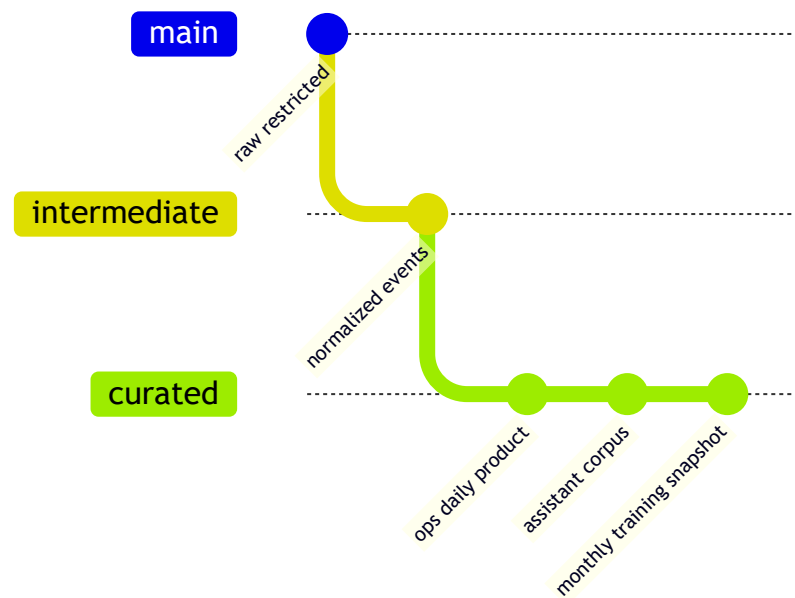
Minimum run evidence per source: source identifier, run date, extraction start and end timestamps, file names or API request parameters, expected vs observed object counts, row counts, checksum or hash for each raw object, schema snapshot, status, and output locations. For the Carrier API specifically, page count and page token history are mandatory. This evidence is what makes Option 2 operationally credible and kills append-without-boundaries behavior.

Failure handling should differ by product. The partial API response should fail publication of the curated shipment facts and block the assistant corpus because those outputs require trustworthy event timelines. The empty CSV can be quarantined only if the affected depot is labeled and excluded from AI answers; otherwise it should also block publication. Quarantine-and-continue is therefore conditional, not default. That nuance is why Option 4 is useful in part but too dangerous as the overall strategy.

Implementation split: use `Python` for source-specific ingestion, manifest generation, and raw writes; use `SQL` and `dbt` for normalization, deduplication, quality tests, and curated product contracts; use `Markdown` for runbook notes covering dependencies, rerun rules, and failure modes. Replace the notebook workflow with a DAG orchestrator that can parameterize run dates, isolate source tasks, support retries, and publish versioned outputs. `Spark` is optional here; at roughly **120,000** daily shipment rows, it is not required unless PDF text extraction or later enrichment volume grows sharply.

The architecture Maya pushed through

Her cutover recommendation was blunt: no launch on the merged notebook path. Rebuild the core into three storage zones and explicit products.



In the revised layout, raw restricted storage kept the source-aligned API JSON pages, original `CSV files`, and document blobs exactly as received. Intermediate models normalized status codes, parsed depot timestamps, and stamped source lineage. Curated outputs were treated as contracts, not just tables:

Product	Consumer	Grain	Freshness	Allowed use	Key quality rule
<code>shipment_events_daily</code>	Ops dashboard	shipment-stop-event	Daily by 6:00 a.m.	Operational reporting	No duplicate business keys; all required depots labeled
<code>exception_assistant_corpus</code>	Exception managers	sanitized chunk/ document	Daily by 6:15 a.m.	Retrieval only	Sensitive text removed; source completeness above threshold
<code>eta_training_snapshot_2024_09</code>	ML team	shipment	Monthly static	Model training	Versioned raw inputs and transformation

Why it matters: this is where governance becomes practical. Sensitive delivery notes, phone numbers, and licensed PDFs stay restricted in raw. Only approved, documented derivatives cross into the AI-ready curated area.

Operationally, Maya wanted runbooks a new engineer could use without calling her. `Markdown` notes listed input windows, dependency order, rerun steps, quarantine logic, and backfill commands. The orchestrator accepted a date parameter, created versioned output paths, and refused to publish curated tables until completeness checks passed. If a source changed type from integer to string, the run captured the schema drift and stopped before silently widening downstream meaning.

Take the lesson somewhere else

Reflection prompt: Think of a pipeline you know that "works" mainly because no one has asked it to prove what happened on a specific run. What raw evidence, product contract, or rerun control would expose its weak point fastest?

- **Align** ingestion mechanics to real source behavior instead of standardizing too early on one connector pattern.
- **Preserve** source-aligned raw inputs so schema drift, partial extracts, and downstream disputes can be replayed with evidence.
- **Separate** restricted raw storage from AI-ready curated products to balance protection with usability.
- **Publish** curated outputs as contracts with consumer, grain, freshness, quality assumptions, and allowed use.
- **Parameterize** reruns with run boundaries and versioned outputs; never append retries into the same logical batch.
- **Document** failure modes and recovery in operational notes, not decorative summaries, so another engineer can run the system at 6:10 a.m.

Decision Points

Option 1. Keep a single notebook workflow that pulls both the Carrier API and partner CSV files directly into one analytics table before the 6:00 a.m. SLA.

- **Expected impact:** This is the fastest path to ship, but it minimizes run evidence, makes reruns unsafe, and hides source-

specific failure modes behind one success flag.

- **Key risk:** A partial API response or empty CSV can still produce a green run while contaminating the assistant corpus and operations dashboard with mixed-quality data.
- **Evidence to confirm:** Three consecutive weeks of runs show stable source counts, deterministic reruns, and no divergence between raw extracts and curated outputs.
- **Evidence to kill:** Any rerun that changes yesterday's totals without a corresponding source correction or versioned run boundary invalidates this approach.

Option 2. Split ingestion by source type, preserve raw source-aligned extracts in restricted storage, and build a hybrid ELT pipeline with dbt-curated AI-ready outputs plus parameterized reruns.

- **Expected impact:** This adds implementation work up front, but it gives the team auditable raw snapshots, consumer-specific curated products, and reproducible recovery under failed overnight runs.
- **Key risk:** The team may miss the launch date if orchestration, contracts, and governance gates are overbuilt for a modest daily volume of about 120,000 shipment rows.
- **Evidence to confirm:** A backfill of seven days can be rerun from preserved raw data with identical curated outputs except for documented source corrections.
- **Evidence to kill:** If raw retention, manifests, and versioned outputs do not materially reduce mean time to recovery during the next schema drift or partial-result incident, the extra complexity is unjustified.

Option 3. Use API ingestion for everything by requiring partners to upload CSV data through an API wrapper so the team can standardize one access pattern.

- **Expected impact:** Operational code paths become more uniform, but the platform inherits API rate limits and may lose file-level evidence such as delivered filenames and exact original payloads.
- **Key risk:** Forcing a single method ignores real source behavior and can create fragile adapters that fail at the worst time during the 5:30 a.m. delivery window.

- **Evidence to confirm:** Wrapped file deliveries preserve immutable original blobs, delivery manifests, and complete row parity with prior CSV-based ingestion.
- **Evidence to kill:** If partners continue sending late or malformed files and the wrapper only converts those failures into opaque HTTP 200 responses, this option should be rejected.

Option 4. Prioritize downstream availability by quarantining suspect records and continuing the pipeline, then repair data later if the dashboard and assistant stay online.

- **Expected impact:** Consumer-facing systems remain fresh more often, especially for dashboard users who tolerate daily summaries, while known-bad slices are isolated for follow-up.
- **Key risk:** Without a hard contract on what can be quarantined, the AI retrieval layer may silently lose high-value documents or shipment events and answer confidently from incomplete context.
- **Evidence to confirm:** Quarantine rates stay below 0.5%, affected entities are clearly labeled in curated outputs, and retrieval quality remains within the agreed acceptance band.
- **Evidence to kill:** If quarantined records cluster around critical lanes or premium customers, or if answer quality drops on those cases, continuing the run is worse than failing fast.

Analysis Walkthrough

Structured comparison is the right lens here because the team is not choosing a single tool; they are choosing an operating model. Option 1 wins on speed and loses almost everywhere else that matters for an AI-native platform: trust, reruns, governance, and evidence. The tell is in the incident data, not the architecture diagram. On Sep 18, the notebook reported success in 24 minutes, yet the curated `shipment_events_daily` table was short by 18,406 rows because the Carrier API returned HTTP 200 with a truncated page set. A green checkmark without source-aligned proof is noise.

Option 3 sounds disciplined because it standardizes interfaces, but it solves the wrong problem. The source assessment already says the sources behave differently: partner CSV files arrive in a narrow 5:20-5:35 a.m. batch window and the Carrier API has pagination and rate-limit quirks. Wrapping files in an API does not make them API-native; it mostly relocates the operational risk.

The module's core lesson shows up plainly here: implementation should align to source characteristics, not force one pattern everywhere.

Option 4 is tempting because HarborLens has two consumer classes with different tolerance. The operations dashboard can accept a daily refresh and some clearly labeled incompleteness. The retrieval assistant used by exception managers cannot. If a lane's exception PDFs or status events are missing, the assistant still answers, just worse and with false confidence. Quarantine-and-continue is viable only for curated products whose contract explicitly permits partial delivery. It is not acceptable as the default for a retrieval corpus unless missing slices are surfaced and blocked from unsafe use.

That leaves Option 2, which I would pick. The evidence supports a source-split ingestion design: CSV files land as immutable objects in restricted raw storage with filename, delivery timestamp, row count, SHA-256, and schema snapshot; the Carrier API lands as raw JSON page payloads plus request parameters, page counts, and completion status. Transformations then run in a hybrid pattern: light normalization at ingest to stamp run metadata, heavier shaping and business rules in SQL and dbt over the raw zone, and curated outputs published for distinct consumers.

The storage decision matters because HarborLens is serving both humans and AI. Restricted raw storage protects sensitive fields like consignee phone numbers and free-text delivery notes while preserving replayability. The AI-ready curated area can then expose sanitized shipment timelines, normalized status codes, and approved document excerpts. This is the BlueRail pattern from this module's Lesson 1 video in practical form: raw, intermediate, and curated each have a purpose, not just a folder name.

The transformation choice should also be hybrid, not pure ETL or pure ELT. Pure ETL would push too much source-specific parsing into Python and make data proofs harder to inspect. Pure ELT would leave too much messy variation in the warehouse entry point, especially when the Carrier API occasionally flips `stop_sequence` from integer to string. A hybrid approach keeps ingestion thin but evidence-rich, then uses dbt models for documented shaping, tests, and curated contracts. That is the easiest way to publish one daily dashboard table and one static monthly training snapshot from the same preserved raw inputs.

The orchestration answer is similarly pragmatic. A simple `Make`-driven local script was enough for the pilot, but the incident history shows the team has outgrown it. They need an explicit DAG orchestrator with parameterized run dates, source-level task boundaries, retry policy for API pages, and rerun rules that never append without a run boundary. The FalconWorks lesson from this module's Lesson 3 video applies directly: another engineer should be able to recover the overnight pipeline at 6:10 a.m. from run artifacts and Markdown notes, not from tribal memory.

The tradeoff I am accepting with Option 2 is more up-front delivery work. The team will spend an extra 2-3 weeks defining manifests, storage zones, dbt tests, and product contracts. That is acceptable because the current failure mode is silent corruption, not visible outage. Silent corruption is worse for an AI-native system: the dashboard may be wrong, the training snapshot may be unreproducible, and the retrieval assistant may be confidently incomplete.

The chart below makes the priority obvious: successful job counts stayed high while trustworthy row parity did not.

